Technical Design Summary

Zijing Liu (Kevin)

Solo project: Mannequin Escape

https://github.com/KevinZijingLiu16/KevinMannequinEscape

Build link: https://kevinzijingliu16.itch.io/mannequin-escape

Final Project Core Mechanics

- Player Control (AGP Sprint Week)
- Inventory (AGP Sprint Week)
- Game Manager (AGP Sprint Week)

- Enemy Sense (AI in Games Sprint Week)
- Enemy State Machine (AI in Games Sprint Week)

Model & Animation (3DANI Sprint Week)

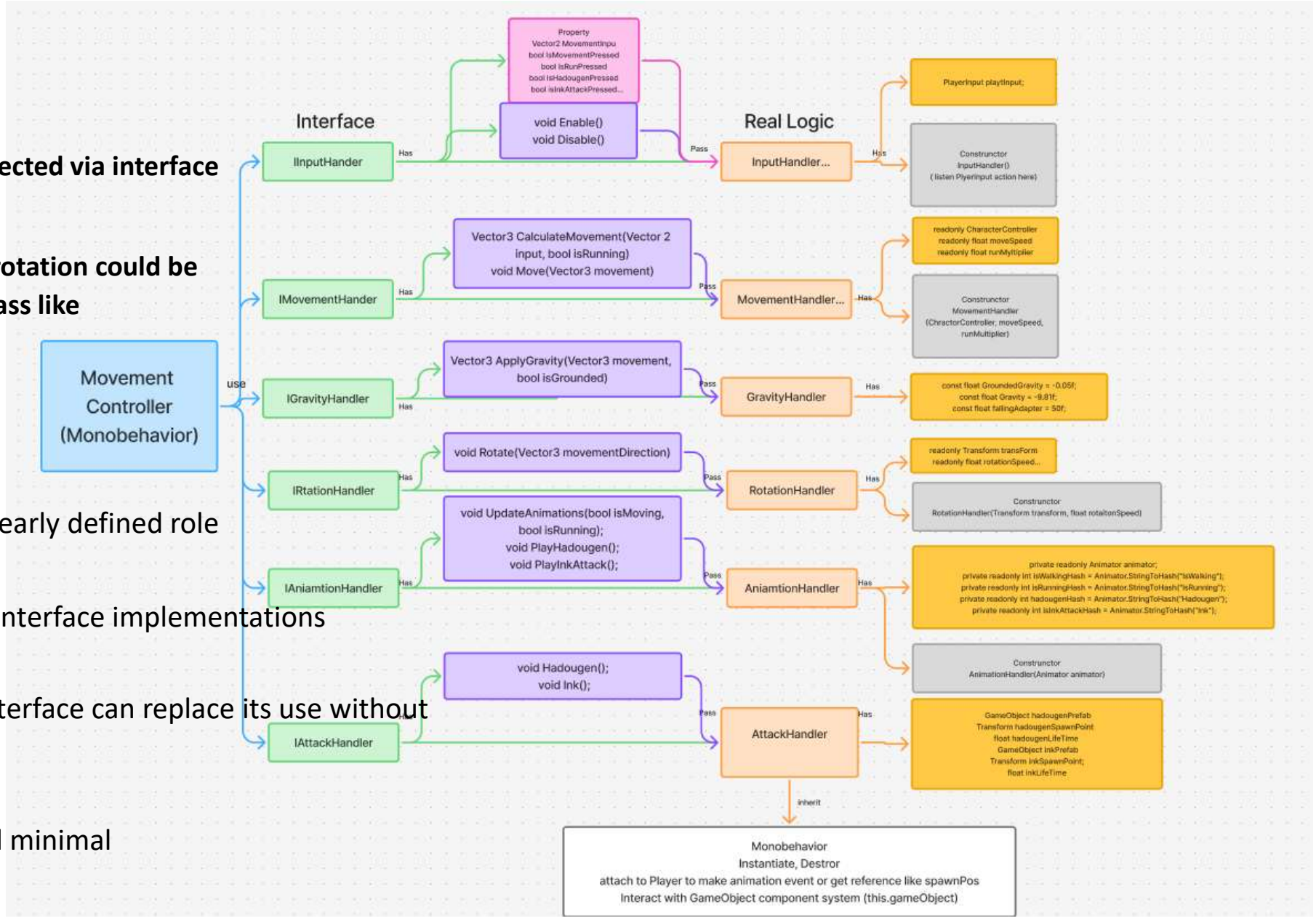
Figma Design Board:

https://www.figma.com/board/AXBljf8bq5lVHqLwYM3JYQ/MannequinEscapeTechDesign?node-id=0-1&p=f&t=L9yGvajEyJqbK97C-0

Player Control

Design Pattern:

- **Component Pattern:**
  - **Modular game behavior injected via interface**
- **Strategy Pattern:**
  - **Movement, input, gravity, rotation could be totally replaced by other class like ZeroGravityHandler.cs**



S - Single Responsibility

- Each class/module has one clearly defined role

O - Open/Closed

- Can extend by swapping out interface implementations

L - Liskov Substitution

- Any class implementing an interface can replace its use without breaking the system

I - Interface Segregation

- All interfaces are focused and minimal

D - Dependency Inversion

- High-Level MovementController depends only on interfaces

Inventory system

Design Pattern:

- ✓ **Model – View – Controller Pattern**
  - Model: InventoryModel
  - View: InventoryUISlot, UIDragIcon, ItemTooltip
  - Controller: InventoryUIController

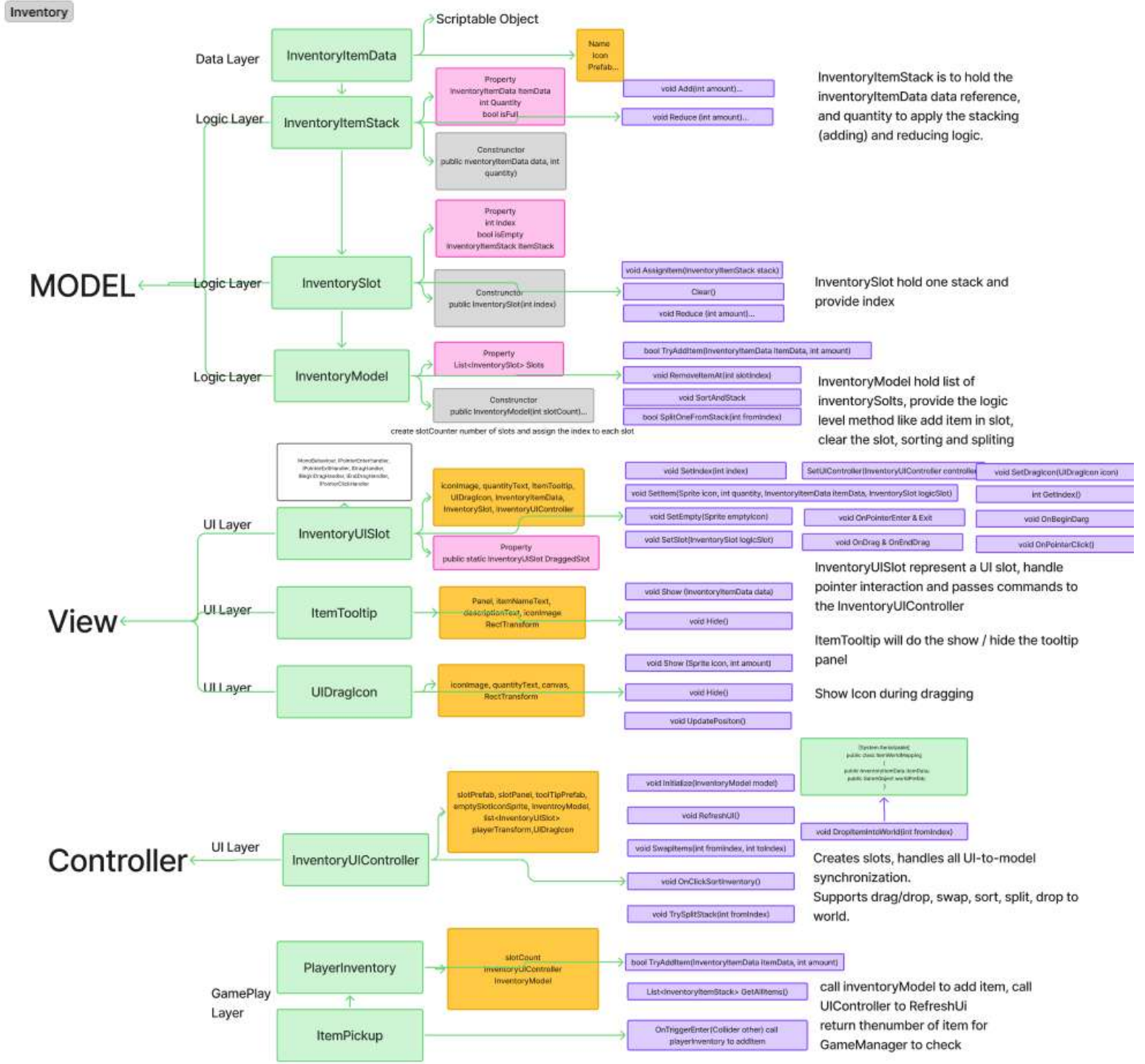
- ✓ **Composite Pattern**
  - InventoryModel holds a List<InventorySlot> which holds InventoryItemStack

Single Responsibility:

- Each class does only one job, item data, stack, slot, UI

Open/Closed:

- Easy to extend like adding another equipment page without modifying the core inventory logic



Game Manager

Design Pattern:

✓ **Observer Event Pattern**

- GameManagers listens events from GameTimer.onTimeOut;
- ItemObjectiveChecker.OnObjectiveCompleted;
- GrabTrigger.OnPlayerGrabbed

Single Responsibility:

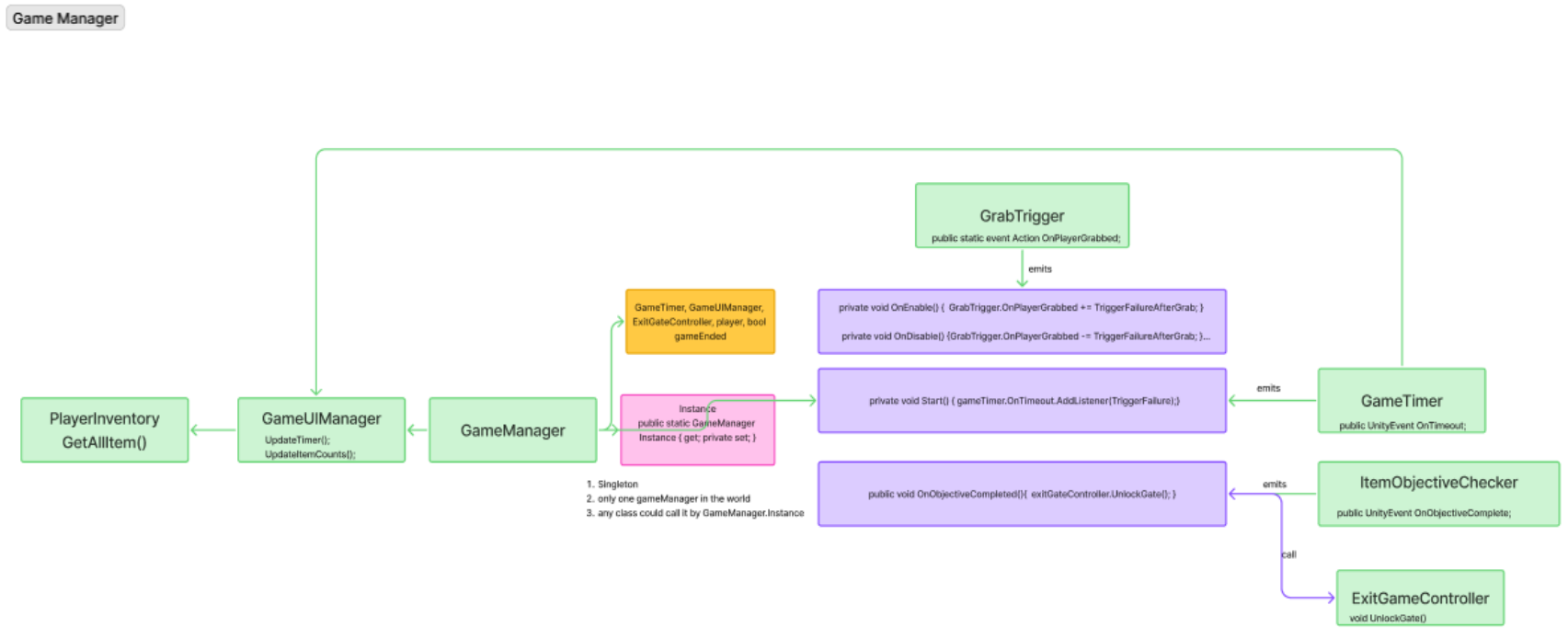
- Each class does only one job, timer, UI, objective tracking, gate control

Open/Closed:

- Easy to extend like adding more winning or losing condition by letting game manager to listen events

Dependency Inversion:

- Game Manager depends on events



[GameManager] (Singleton)
- Controls [GameTimer]
- Controls [ExitGateController]
- Controls [GameUIManager]
- Listens to [GrabTrigger events] (external event system)

[GameTimer]
- Emits [OnTimeout UnityEvent] → GameManager listens

[ExitGateController]
- Can be [Unlocked] → Detects [Player] to trigger victory

[GameUIManager]
- Displays Timer
- Displays in Game item counts (Cube, Sphere, Capsule)
- Handles Win/Loss UI Panels

[ItemObjectiveChecker]
- Watches Inventory to detect if goal is met → Emits [OnObjectiveComplete] event

Enemy Perception

Design Pattern:

✓ **Observer Event Pattern**

- Perception Component will listen events from sense components
- Hit and Noise Sense will listen to OnHit and OnNoiseEmitted events

✓ **Template Method**

- SenseComponent defines sensing framework, AlwaysAwareSense, SightSense, HitSense and NoiseSense override behaviour

Single Responsibility:

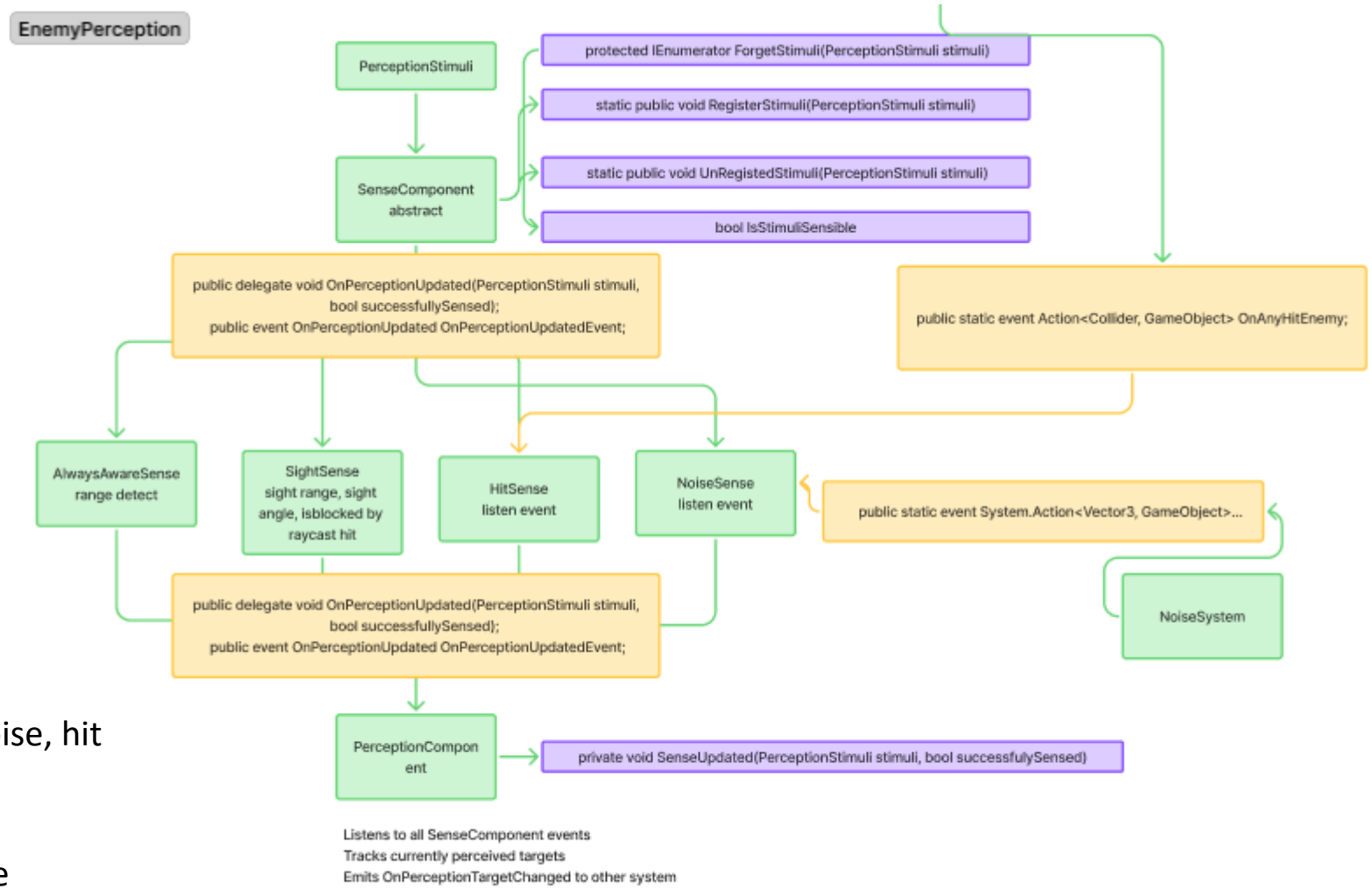
- Each class does only one job, sight, noise, hit etc..

Open/Closed:

- Easy to add new sense like smell sense

Dependency Inversion:

- PerceptionComponent depends on event from senses



Enemy State machine

Design Pattern:

- ✓ **State Pattern**
- ✓ **Template Method**
  - State, StateMachine, EnemyBaseState areabstract class to provide the basic framwork

Single Responsibility:

- Each state handle their own stuff like animation

Open/Closed:

- Easy to add new state like BlockingState

